Member-only story

Data Engineering + | Data Science ✓ | Etl + | Data Analysis +

10 GenAI Use Cases in ETL You Can Implement Today

Automation and insight by applying GenAI inside your ETL stack



Rohan Dutt Follow 8 min read · Nov 9, 2025

173 3

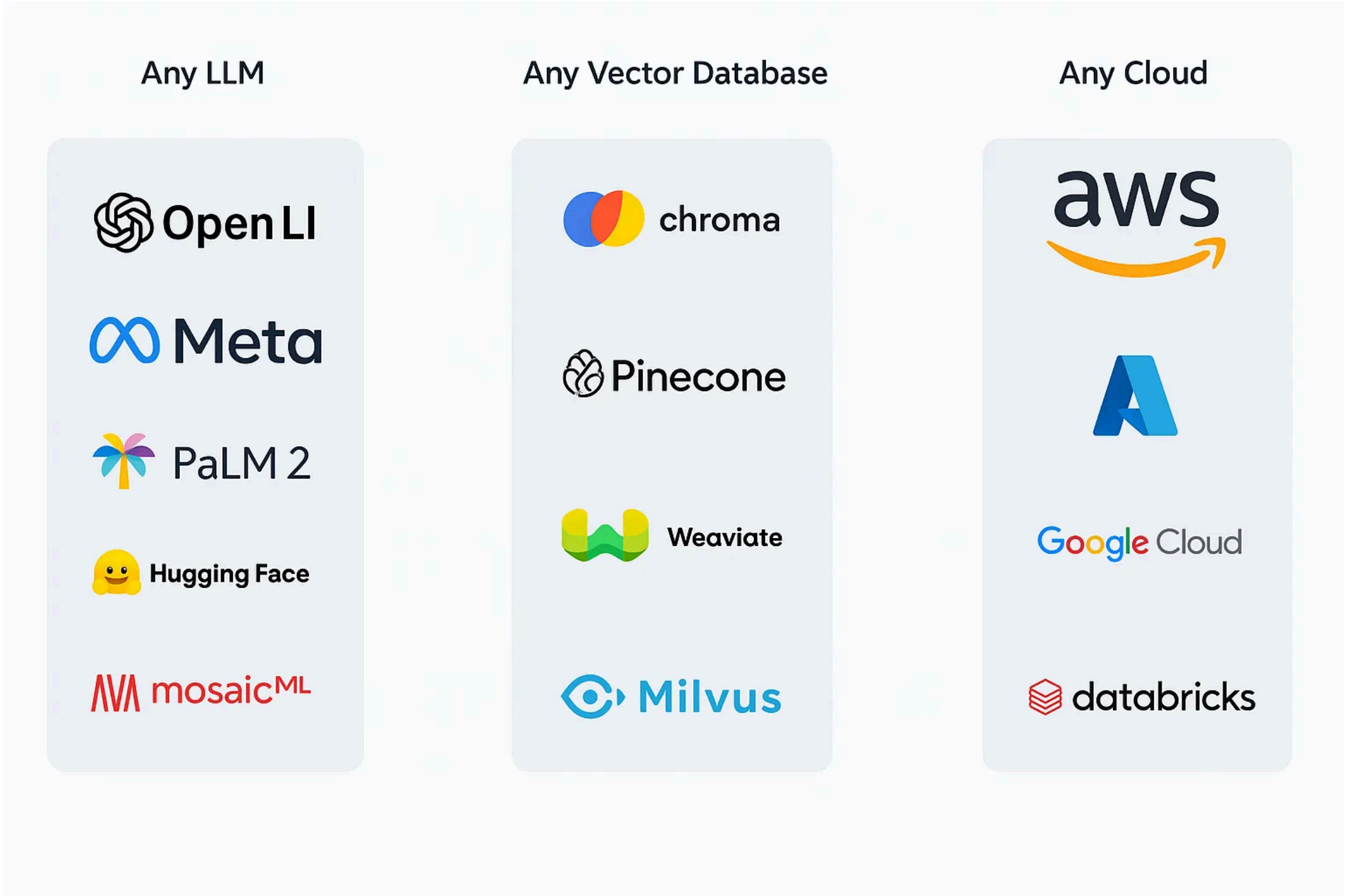

Any LLM
OpenLI
Meta
PaLM 2
Hugging Face
mosaicML
Any Vector Database
chroma
Pinecone
Weaviate
Milvus
Any Cloud
aws
Google Cloud
databricks

Image by Dr. Rabi Prasad Padhy

*— **Non Member**: Pls take a look **here**!*

GenAI use cases that enhance ETL workflows immediately. These examples show how AI can automate, optimize, and simplify data pipelines.

10. Auto-Generate Schema Mappings with Natural Language

Why it matters: Manually mapping schemas eats up 30% of ETL development time. GenAI can interpret source/target structures and suggest mappings, just describe them in plain English.

How to use it:

- Feed sample data + a brief description (e.g., *"Match `customer_name` in CSV to `client_name` in SQL"*) to GenAI tools like ChatGPT or Google DataPlex.
- Validate outputs with a lightweight test pipeline before full deployment.

Example:
Suppose you have a CSV with columns `customer_name`, `email_address`, and

`signup_date`, and a SQL table with columns `client_name`, `email`, and `created_at`. You could prompt:

"Map CSV columns to SQL table columns: `customer_name` → `client_name`, `email_address` → `email`, `signup_date` → `created_at`."

The AI tool can then generate a mapping table like:

```
| Source CSV    | Target SQL  | Transformation            |
| ------------- | ----------- | ------------------------- |
| customer_name | client_name | None                      |
| email_address | email       | Lowercase for consistency |
| signup_date   | created_at  | Convert to YYYY-MM-DD     |
```

💡 **Try this:** Use **LlamaIndex** to auto-document mapping logic so future audits or schema changes can be tracked automatically.

9. Fix Dirty Data Before It Hits Your Pipeline

Why it matters: Bad data costs businesses significant processing time and cost. GenAI can scrub duplicates, standardize formats, and flag anomalies in real time.

How to use it:

- Deploy Python libraries like **Cleanlab** with LLM-powered rules (e.g., *"Standardize all EU dates to YYYY-MM-DD"* or *"Ensure phone numbers are in E.164 format"*).
- For streaming data, integrate **Apache Kafka** with AI validation hooks to automatically catch anomalies as events flow in.

Example:
Suppose a CSV stream contains `product_id`, `SKU`, and `price`, but some SKUs are misformatted or duplicated. You can define an LLM prompt:

"Identify duplicate SKUs, standardize formats to uppercase, and flag prices that deviate >20% from historical median."

The AI can output:

```
| product_id | SKU   | price | action               |
| ---------- | ----- | ----- | -------------------- |
| 101        | ab123 | 25    | Correct SKU → AB123  |
```

```
| 102        | AB123 | 25    | Duplicate → merge    |
| 103        | XY999 | 500   | Price anomaly → flag |
```

8. Translate Legacy SQL to Modern Syntax Automatically

Why it matters: Many ETL pipelines still rely on older SQL dialects (e.g., SQL-92, older Oracle/DB2 syntax). For example, the tool Lakebridge from Databricks can automate migration of legacy queries and claims to convert “up to 80% of migration tasks … so teams can move faster”.

How to use it:

- Paste your legacy SQL query into a GenAI tool (e.g., Codex, Claude 3), specify the target platform (e.g., Spark SQL, BigQuery SQL, or Snowflake SQL).
- The AI suggests a transformed query with updated functions/syntax.
- Always validate the converted query in a staging environment to confirm logic and performance.

Example:
Suppose you have a legacy Oracle query such as:

```
SELECT TO_CHAR(orders.date_entered,'DD-MON-YYYY') AS order_date,
       customer.name AS cust_name,
       SUM(order_items.qty * order_items.unit_price) AS total_value
FROM orders
JOIN customer ON orders.customer_id = customer.customer_id
JOIN order_items ON order_items.order_id = orders.order_id
WHERE orders.status = 'COMPLETE'
GROUP BY TO_CHAR(orders.date_entered,'DD-MON-YYYY'),
         customer.name
HAVING SUM(order_items.qty * order_items.unit_price) > 1000;
```

You ask the AI:

> “Convert this Oracle query into Spark SQL dialect for use in Databricks, retaining the same business logic and grouping.”

The tool returns something like:

```
SELECT date_format(orders.date_entered, 'yyyy-MM-dd') AS order_date,
       customer.name AS cust_name,
       SUM(order_items.qty * order_items.unit_price) AS total_value
FROM orders
JOIN customer ON orders.customer_id = customer.customer_id
JOIN order_items ON order_items.order_id = orders.order_id
WHERE orders.status = 'COMPLETE'
```

```
GROUP BY date_format(orders.date_entered, 'yyyy-MM-dd'),
         customer.name
HAVING SUM(order_items.qty * order_items.unit_price) > 1000
```

Key changes & benefits:

- `TO_CHAR(...,'DD-MON-YYYY')` replaced with `date_format(..., 'yyyy-MM-dd')` to align with Spark SQL standards.
- Syntax remains equivalent, so business logic is preserved.
- Faster conversion, fewer manual edits — especially useful when hundreds of such queries exist.

📊 **Stat:** One source notes that AI-powered conversion can reduce migration time significantly ("Automating thousands of SQL scripts into modern code … 70 % faster" in a case study).

7. Generate Synthetic Test Data On-Demand

Why it matters: Testing pipelines with production data is risky — and manually creating mock data is time-consuming. GenAI lets you generate realistic, GDPR-compliant synthetic datasets instantly.

How to use it:

- Prompt tools like **Gretel.ai** to create synthetic datasets. For example:

> "Generate 10,000 rows mimicking our `customer` table, but randomize names, emails, and phone numbers while keeping realistic distributions for age and signup dates."

- Validate the synthetic data using statistical similarity checks (mean, median, distribution shape) to ensure it reflects the production dataset's characteristics without exposing real PII.

Example:
Suppose your production `customer` table has:

```
| customer_id | name  | email                                | signup_date | t
| ----------- | ----- | ------------------------------------ | ----------- | -
| 101         | Alice | [alice@example.com](alice@example.com) | 2023-01-12  | 2
| 102         | Bob   | [bob@example.com](bob@example.com)     | 2023-02-08  | 1
```

A synthetic generation prompt could be:

> *"Generate 10,000 rows that mimic the schema above, replacing names/emails with realistic fakes, keeping total_spent between 0–5000, and include some null signup_dates for testing."*

The AI outputs:

customer_id	name	email	signup_date
1	Emma K.	emma.k@fakesite.io	2023-03-01
2	Liam T.	liam.t@fakesite.io	NULL
3	Zoe L.	zoe.l@fakesite.io	2023-01-20

💡 **Try this:** Seed edge cases intentionally, such as null values, extreme totals, or negative values, to stress-test validations, aggregations, and error-handling in your ETL pipelines.

6. Auto-Document Lineage & Metadata

Why it matters: 47% of data teams can't trace column-level lineage (Monte Carlo). GenAI can automatically parse pipeline code and generate interactive documentation, saving hours of manual effort.

How to use it:

- Point **OpenMetadata** or **Marquez** at your DAGs. Their AI agents can automatically tag:
- **Sources** (e.g., raw tables or APIs)
- **Transformations** (e.g., joins, aggregations, feature engineering)
- **Owners** (who is responsible for each dataset)
- Embed "Explain this transformation" buttons in internal documentation so analysts and auditors can get natural-language explanations instantly.

Example:
Suppose your ETL pipeline transforms a `sales_raw` table into `sales_clean` with several steps:

1. Remove null `customer_id` rows
2. Standardize `country_code` to ISO format

3. Aggregate daily sales per store

The AI can automatically generate a lineage table like:

```
| Target Table | Source Table | Transformation Description
| ------------ | ------------ | ----------------------------------------------
| sales_clean  | sales_raw    | Drop null customer_id, standardize `country_code
```

A user clicking **“Explain this transformation”** could see:

> *“This step removes incomplete records, converts country codes to ISO standard, and sums sales for each store per day.”*

🔥 **Real-world impact:** A bank reduced compliance audit prep from **weeks to hours** by using AI-generated lineage maps to automatically produce documentation for dozens of pipelines.

5. Optimize Orchestration with AI-Powered Scheduling

Why it matters: Static cron jobs and fixed schedules waste resources and miss optimisation opportunities. AI-driven scheduling analyses historical runtimes, dependencies and cost signals to dynamically adjust when jobs run, saving cloud cost and improving throughput.

How to use it:

- Use orchestration tools (e.g., Apache Airflow combined with platforms like Astronomer) enhanced with ML/AI-capabilities to predict task back-log, resource contention and delays, and adjust task schedules accordingly (e.g., “run this heavy job between 02:00–04:00 when spot instances are cheaper”).
- Define cost-aware rules: for example — *“Delay non-urgent jobs if cloud cost spike > 15% or available quota drops below threshold”*.
- Continuously feed historical task runtimes, dependency graphs, resource usage and cost metrics into your model to refine scheduling decisions and detect when to shift tasks to off-peak windows or scale differently.

Example:
Imagine you have a daily pipeline with three tasks:

- Task A (ingest raw data) runs at 01:00, takes ~20 minutes.
- Task B (heavy transformation) runs at 01:30, takes ~2 hours and uses high-memory instances.
- Task C (load summary) runs at 04:00, takes ~30 minutes.

Over time, you notice: on weekends, Task B often finishes early because upstream data is smaller; on weekdays holiday periods it gets delayed because of shared resource contention.

You prompt your AI-scheduler:

> "Analyse past 90 days runtimes for tasks A, B, C; identify patterns of contention; propose optimal windows to run heavy jobs to minimise cost and resource wait time."

The tool responds with suggestions like:

- On weekends/low-volume days: shift Task B to start at 00:45 (instead of 01:30) using cheaper spot instances, finishing by ~01:30, freeing up normal instance capacity for other jobs.

- On high-volume weekdays: if cloud cost index (spot price + instance demand) is above threshold at 01:00, defer Task B to 02:15 when costs drop ~12%.
- On non-urgent tasks (e.g., reports) spawn them later in the night when costs fall, rather than during peak.

Outcome & benefit:

- Reduced idle resource time for heavy transformation jobs.
- Shifting non-critical tasks to cheaper windows cut cloud compute spend.
- Improved pipeline reliability: fewer unexpected resource-contention delays.
- According to industry commentary: AI orchestration and dynamic scheduling deliver operational cost savings (~20-30%) in cloud workloads.

Medium Search Play Games Write

delivering ≈ **25-30% cost savings** in multi-cloud environments. Nuroblox+1

4. Convert Business Rules to Data Transformations Instantly

Why it matters: Stakeholders describe logic in plain English (“Apply a 10% discount to repeat customers”), forcing devs to manually code it.

How to use it:

- Feed rules to LLMs fine-tuned on your data model (e.g., “Generate PySpark code for this promo logic”).
- Pair with human-in-the-loop validation for complex logic.

💡 Try this: Store approved rule-to-code pairs in a vector DB to accelerate future requests.

3. Auto-Heal Broken Pipelines

Why it matters: 80% of pipeline failures follow predictable patterns (schema drift, timeout, etc.).

How to use it:

- Deploy Monte Carlo or Anomalo to detect failures and suggest fixes (e.g., “Source column ‘price’ disappeared, use ‘unit_cost’ instead?”).

- For custom setups, train GPT-4 on past incident reports to recommend solutions.

🔥 Real-world example: A logistics company reduced MTTR (mean time to repair) by 65% with AI-powered alerts.

2. Predict & Scale Resources Before Bottlenecks Hit

Why it matters: Over-provisioning wastes money; under-provisioning kills SLAs.

How to use it:

- Tools like Databricks AutoML forecast load spikes based on historical trends.
- Set auto-scaling triggers (e.g., “Add 2 workers if CPU > 75% for 5 mins”).

📊 Stat: AI-driven scaling reduces idle compute costs by 40% (Google Cloud).

1. Turn Natural Language into Full ETL Pipelines

Why it matters: Describe workflows in plain text (“Ingest Shopify orders, join with CRM, aggregate by region”), and GenAI assembles the pipeline.

How to use it:

- Platforms like Defog.ai and Pandas AI already prototype this.
- Start small, use AI to generate boilerplate code, then refine manually.

🚀 Pro tip: Pair with CI/CD checks to ensure AI-generated pipelines meet security/compliance rules.

Thankyou... Clap 50 times and Follow for more :)

Data Engineering + | Data Science ✓ | Etl + | Data Analysis +



Written by Rohan Dutt

3.1K followers · 84 following

I am a data scientist passionate about AI and its potentials. And love to write about my experiences in the tech industry and latest tech developments.

Follow

Help Status About Careers Press Blog Store Privacy Rules Terms Text to speech